DOPPLER THINGS

COMPUTER PROJECT MANAGEMENT
DSP-BASED PROJECT MANAGED USING CPM

SUBMISSION DATE: JANUARY 14, 2026

PROJECT NAME:

Gesture Control Using Doppler Shift Detection

MEMBERS:

Logie, Mendiola
Mazaredo, Jasmin Paige T.
Parrish, Paulean Marguerette F.

DSP TASK	PROJECT MANAGEMENT ASPECT
• Signal Analysis	• Requirement Gathering
• Technical Planning	• Technical Planning
• Execution	• Execution
• Schedule & Cost Control	• Schedule & Cost Control
• Quality Assurance	• Quality Assurance
• Project Closure	• Project Closure

PROJECT OVERVIEW

Gesture Control Using Doppler Shift Detection is a DSP-based interactive system that detects simple hand movements by analyzing Doppler frequency shifts produced when a hand moves toward or away from a sensor. The project aims to demonstrate how motion can be detected and interpreted using signal processing techniques without physical contact, emphasizing low-cost implementation and real-time DSP analysis.

The system operates by transmitting a continuous ultrasonic signal and receiving its reflected waveform. When a user moves their hand relative to the sensor, the reflected signal experiences a slight frequency shift due to the Doppler effect. This shift is directly related to the direction and speed of the hand movement. By processing the received signal in the frequency domain, the system can identify whether the hand is moving toward or away from the sensor.
Detected gestures are mapped to a simple control interface, such as increasing or decreasing a virtual volume level, moving a graphical slider, or controlling the brightness of an indicator. A hand moving toward the sensor results in an increase in the controlled parameter, while movement away causes a decrease. This interaction provides immediate visual feedback, making the system intuitive and engaging.

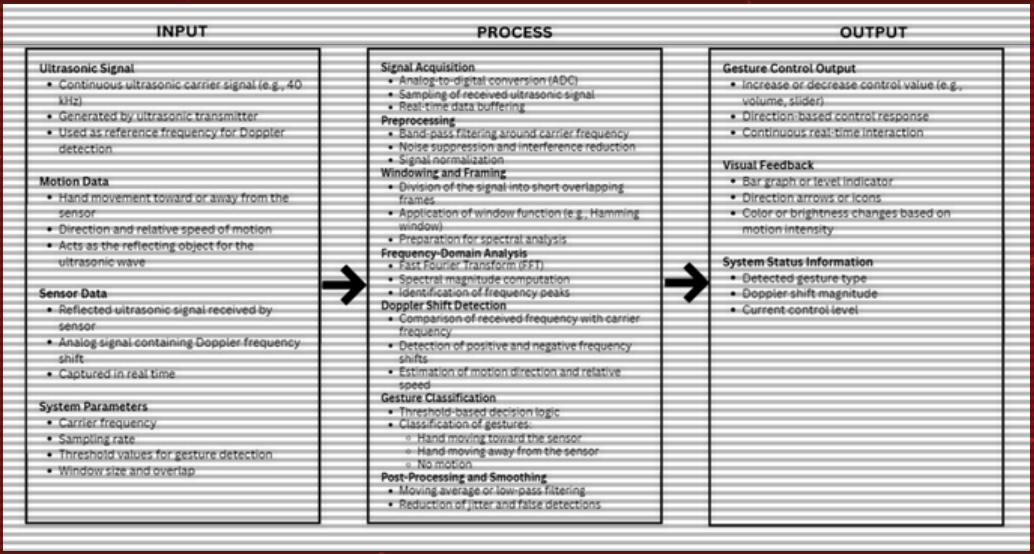
The project focuses primarily on digital signal processing concepts rather than complex hardware. The Doppler effect may be implemented through simulation or using low-cost ultrasonic transducers, allowing the project to remain feasible within a limited budget and development timeframe. Despite its simplicity, the system effectively demonstrates real-time spectral analysis, frequency estimation, and motion-based human–computer interaction using DSP principles.

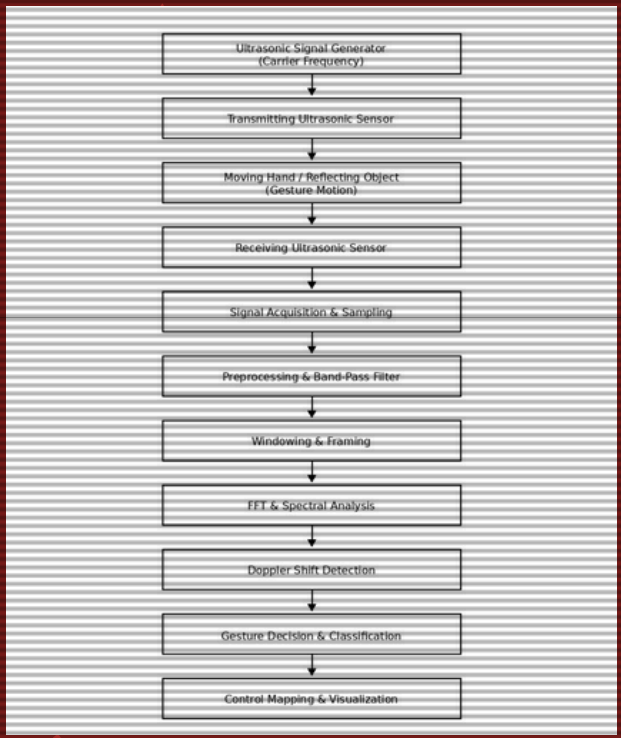
SIGNAL ANALYSIS/REQUIREMENT GATHERING

The initial phase of signal analysis directly corresponds to the requirement gathering stage of project management. In this phase, the fundamental requirements for detecting hand gestures using ultrasonic sensors were identified and documented. The HC-SR04 ultrasonic sensors were selected as the primary signal acquisition hardware, operating at 40 kHz frequency to emit and receive ultrasonic pulses. The requirement gathering process involved determining the necessary detection range, identifying the types of gestures to be recognized (hand movements toward and away from the sensors), and establishing the environmental constraints under which the system must operate.

This phase also included stakeholder analysis, where the end-user experience requirements were defined, particularly focusing on creating an immersive Stranger Things atmosphere where gesture controls would feel intuitive and responsive. The technical requirements specified the need for real-time signal processing capabilities, low-latency response times suitable for gaming applications, and reliable gesture recognition accuracy even in varying environmental

FILTER DESIGN/TECHNICAL PLANNING





Following requirement identification, the filter design phase aligned with technical planning in project management. This stage involved developing the signal processing architecture necessary to extract meaningful gesture information from raw ultrasonic sensor data. A moving average filter was designed as the primary noise reduction technique, configured to smooth the incoming distance measurements while maintaining sufficient temporal resolution to capture dynamic hand movements.

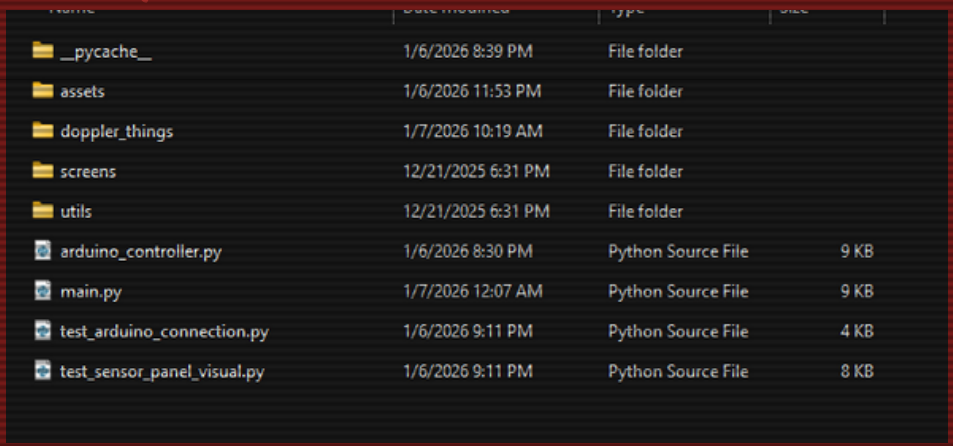
The technical planning included selecting appropriate filter window sizes, determining sampling rates adequate for capturing the Doppler effect caused by hand motion, and architecting the data flow from sensor input through processing stages to final game control outputs.

During this phase, the Arduino code structure was planned, including the organization of functions for sensor reading, signal filtering, threshold validation, and serial communication protocols. The technical planning also encompassed designing the integration between the Arduino hardware system and the Python game engine, establishing the communication protocol through serial port data transmission, and planning the coordinate mapping system that would translate physical gesture positions into virtual game world interactions.

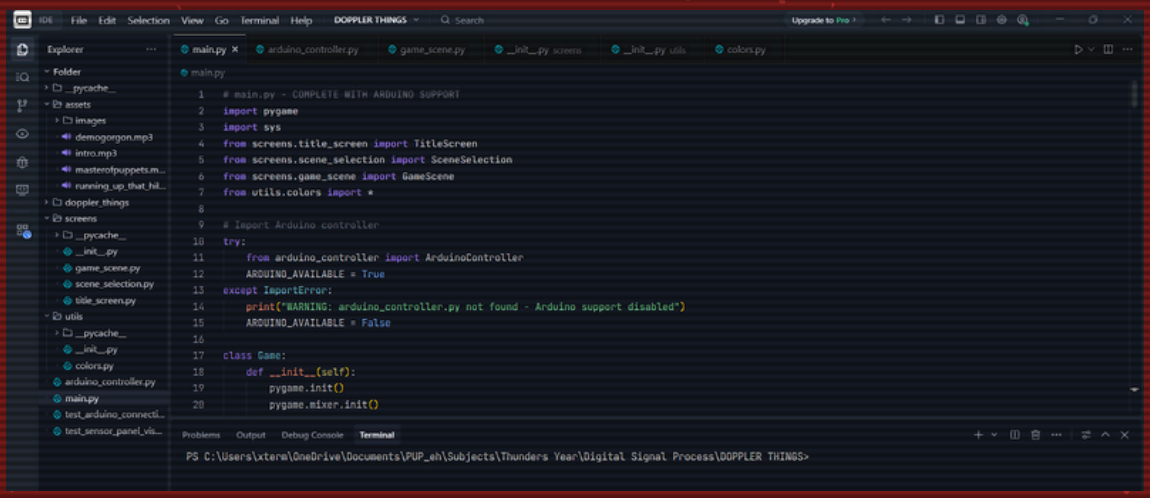
ALGORITHM CODING / EXECUTION

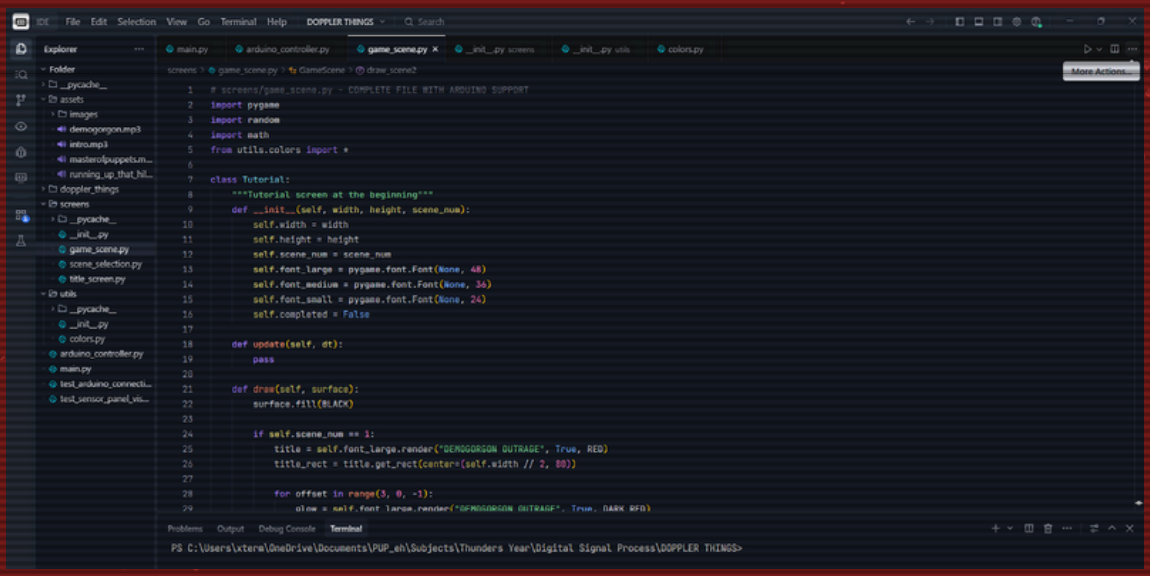
The execution phase manifested through algorithm coding, where the designed signal processing techniques were implemented in Arduino IDE and Python, specifically PyGame. The core Doppler shift detection algorithm was coded to analyze the rate of change in distance measurements, identifying approaching and receding hand movements based on velocity calculations derived from consecutive sensor readings. Implementation included threshold-based validation logic to distinguish intentional gestures from environmental noise or unintended movements, ensuring robust gesture recognition.

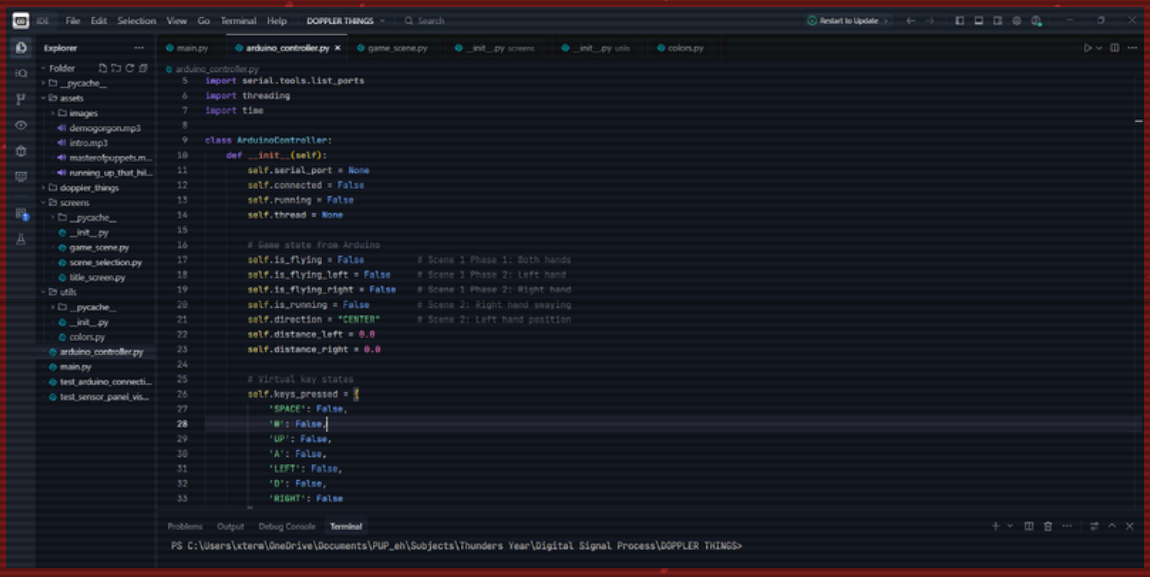
The moving average filter was coded with configurable parameters allowing real-time adjustment of smoothing characteristics based on performance testing feedback. Serial communication protocols were implemented to transmit processed gesture data from the Arduino to the Python game environment, with appropriate data formatting and error checking to ensure reliable transmission. Within the Python environment, game control scripts were developed to receive serial data, parse gesture commands, and translate them into player actions within the Stranger Things-themed game world. The execution phase also involved coding the visual and audio elements that provide feedback to users, creating an immersive experience where Christmas light sequences flash and audio cues respond to detected gestures, maintaining thematic consistency with the Stranger Things narrative.

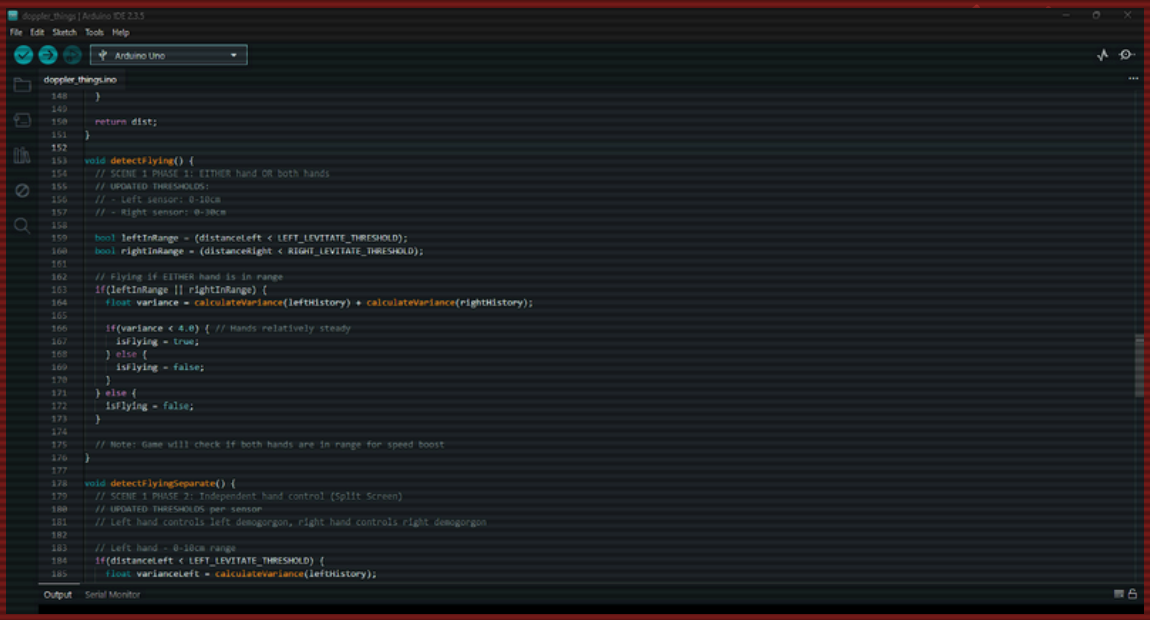


Name	Date modified	Type	Size
__pycache__	1/6/2026 8:39 PM	File folder	
assets	1/6/2026 11:53 PM	File folder	
doppler_things	1/7/2026 10:19 AM	File folder	
screens	12/21/2025 6:31 PM	File folder	
utils	12/21/2025 6:31 PM	File folder	
arduino_controller.py	1/6/2026 8:30 PM	Python Source File	9 KB
main.py	1/7/2026 12:07 AM	Python Source File	9 KB
test_arduino_connection.py	1/6/2026 9:11 PM	Python Source File	4 KB
test_sensor_panel_visual.py	1/6/2026 9:11 PM	Python Source File	8 KB


DOPPLER THINGS
main.py
arduino_controller.py
game_scene.py
colors.py
main.py - COMPLETE WITH ARDUINO SUPPORT
import pygame
import sys
from screens.title_screen import TitleScreen
from screens.scene_selection import SceneSelection
from screens.game_scene import GameScene
from utils.colors import *
Import Arduino controller
try:
from arduino_controller import ArduinoController
ARDUINO_AVAILABLE = True
except ImportError:
print("WARNING: arduino_controller.py not found - Arduino support disabled")
ARDUINO_AVAILABLE = False
class Game:
def __init__(self):
pygame.init()
pygame.mixer.init()
Terminal
PS C:\Users\xterm\OneDrive\Documents\PUP_eh\Subjects\Thunders Year\Digital Signal Process\DOPPLER THINGS>


DOPPLER THINGS
game_scene.py
screens/game_scene.py - COMPLETE FILE WITH ARDUINO SUPPORT
import pygame
import random
import math
from utils.colors import *
class Tutorial:
"""Tutorial screen at the beginning"""
def __init__(self, width, height, scene_num):
self.width = width
self.height = height
self.scene_num = scene_num
self.font_large = pygame.font.Font(None, 48)
self.font_medium = pygame.font.Font(None, 36)
self.font_small = pygame.font.Font(None, 24)
self.completed = False
def update(self, dt):
pass
def draw(self, surface):
surface.fill(BLACK)
if self.scene_num == 1:
title = self.font_large.render("DEMOGORGON OUTRAGE", True, RED)
title_rect = title.get_rect(center=(self.width // 2, 80))
for offset in range(3, 0, -1):
Terminal
PS C:\Users\xterm\OneDrive\Documents\PUP_eh\Subjects\Thunders Year\Digital Signal Process\DOPPLER THINGS>


DOPPLER THINGS
arduino_controller.py
import serial.tools.list_ports
import threading
import time
class ArduinoController:
def __init__(self):
self.serial_port = None
self.connected = False
self.running = False
self.thread = None
Game state from Arduino
self.is_flying = False # Scene 1 Phase 1: Both hands
self.is_flying_left = False # Scene 1 Phase 2: Left hand
self.is_flying_right = False # Scene 1 Phase 2: Right hand
self.is_running = False # Scene 2: Right hand swaying
self.direction = "CENTER" # Scene 2: Left hand position
self.distance_left = 0.0
self.distance_right = 0.0
Virtual key states
self.keys_pressed = {
'SPACE': False,
'W': False,
'UP': False,
'A': False,
'LEFT': False,
'D': False,
'RIGHT': False
Terminal
PS C:\Users\xterm\OneDrive\Documents\PUP_eh\Subjects\Thunders Year\Digital Signal Process\DOPPLER THINGS>



OPTIMIZATION / SCHEDULE & COST CONTROL

As the implementation progressed, optimization efforts aligned with schedule and cost control project management practices. Signal processing parameters were fine-tuned to minimize latency between gesture execution and game response, critical for maintaining player engagement and gameplay fluidity. Filter coefficients were optimized to balance noise reduction against response time, ensuring gestures were detected quickly without sacrificing accuracy. Memory usage was optimized in both the Arduino firmware and Python game code to ensure smooth performance on target hardware platforms.

From a schedule control perspective, iterative testing cycles were planned and executed to identify performance bottlenecks and address them systematically before the January 7th DSP Fiesta deadline. Cost control was managed by utilizing available Arduino Uno boards and HC-SR04 sensors rather than procuring expensive specialized hardware, demonstrating resourcefulness in achieving project objectives within budget constraints. Development time was optimized by leveraging existing Arduino libraries for sensor interfacing and Python assets for game environment creation, allowing focus on the unique DSP algorithm implementation rather than reinventing common functionality.

PERFORMANCE TESTING / QUALITY ASSURANCE

Quality assurance was conducted through comprehensive performance testing of the gesture recognition system under various conditions. Testing scenarios included varying hand movement speeds to verify the system could detect both slow deliberate gestures and quick reactive movements. Distance range testing validated that gestures were reliably recognized across the intended operating range of the sensors, from close proximity to maximum effective distance.

Environmental testing assessed system performance under different lighting conditions and background noise levels, ensuring robustness in the DSP Fiesta demonstration environment. Accuracy metrics were collected, measuring the gesture recognition success rate and false positive detection frequency to quantify system reliability. User acceptance testing involved having test participants interact with the system, gathering feedback on gesture intuitiveness, response time perception, and overall gaming experience. Quality assurance also included code review processes to identify potential bugs, optimize code structure, and ensure adherence to best practices in embedded systems programming and game development.


Doppler Things
DOPPLER
THINGS
PRESS SPACE OR CLICK TO START


Doppler Things
SCENE SELECTION
DEMOGORGON
OUTRAGE
ESCAPE
VECNA
BACK
Press 1 or 2 for quick select | ESC to go back


Doppler Things
HEALTH
DODGED: 0
DISTANCE: 216m
PORTAL: 4783m
SPEED: 1.0x
Press ESC to go back


Doppler Things
HOLD SPACE to fill meter!
HEALTH: 96
KILLS: 0
SPACE
0%
Press ESC to go back

DEPLOYMENT / PROJECT CLOSURE

The deployment phase culminates in the DSP Fiesta presentation on January 7, 2026, where the Doppler Things project will be demonstrated to faculty, peers, and potentially external evaluators. Deployment preparation includes finalizing the hardware setup with secure sensor mounting, reliable power supply configuration, and backup systems to ensure demonstration reliability. The Python game build is compiled and tested on the presentation hardware to verify compatibility and performance. Documentation is completed, including technical specifications detailing the DSP algorithms implemented, user guides explaining gesture controls, and project reports summarizing the development process aligned with project management principles.

Project closure activities include conducting post-mortem analysis to document lessons learned, archiving project files and code repositories for future reference, and preparing final deliverables including this CPM activity report that demonstrates the successful integration of DSP technical work with project management frameworks. The closure phase also involves reflecting on how each technical DSP task naturally mapped to project management aspects, reinforcing the understanding that engineering projects inherently follow project management structures whether explicitly recognized or not.




HOLD SPACE to fill meter!
SENSORS
ACTIVE






DOPPLER
DOPPLER
DOPPLER

Fin.